

VPA A-Level CS — Weeks 1-9: Programming Fundamentals

AS Level Semester 2 — Weeks 1 - 9

Programming Fundamentals 编程基础 (Biānchéng jīchǔ)

Syllabus Coverage: 2.1 Algorithm Design, 2.2 Data Structures (1D/2D arrays, records), 2.3 Programming (variables, operators, selection, iteration, subroutines, recursion)

Paper: Paper 2 (Fundamental Problem-solving & Programming)

Duration: 9 weeks (Weeks 1 - 9)

Assessment: Baseline test (W1), Quiz (W2), Practical test (W4), Recursion worksheet (W5), Mid-unit test (W6), Array tasks (W7-8), Half-Term Exam (W9)

Week-by-Week Scheme of Work

Week 1: Pseudocode Mastery — Variables, Constants, I/O, Expressions

伪代码精通：变量、常量、输入输出、表达式

Lesson 1 (40 min): Variables, Constants & Data Types 变量、常量与数据类型

Learning Objectives:

- ☐ Declare variables with correct data types 用正确的数据类型声明变量
- ☐ Distinguish between variables and constants 区分变量和常量
- ☐ Select appropriate data types for given data 为给定数据选择合适的数据类型

Starter (5 min): "Do Now" — Given: age=16, price=19.99, name="Alice", isStudent=TRUE. What types are these?

Main Activity (20 min):

- ☐ Variable: Named memory location whose value can change. 变量：命名的内存位置，值可以改变。
- ☐ Constant: Named value that does not change during execution. 常量：执行期间不改变的命名值。
- ☐ CIE data types:
 - INTEGER: Whole numbers (-3, 0, 42). 4 bytes typically.
 - REAL: Decimal numbers (3.14, -0.5). Floating-point representation.
 - STRING: Text ("Hello", "123"). Sequence of characters.
 - CHAR: Single character ('A', '7').
 - BOOLEAN: TRUE or FALSE. 1 byte.
 - DATE: Calendar dates.
- ☐ Declaration syntax: DECLARE Age : INTEGER
- ☐ Constant syntax: CONSTANT Pi = 3.14159
- ☐ Assignment: Age ← 16 (the arrow shows direction of data flow)
- ☐ Naming conventions: Meaningful names, no spaces, CamelCase or underscores.
- ☐ Type errors: Age ← "Hello" → Type mismatch!

Activity (10 min): Given scenario (student registration), students declare 8 variables with correct types.

Plenary (5 min): "Why use constants instead of hard-coding values?" Discuss: maintainability, readability, single point of change.

Homework: Declare variables for a library book system: title, author, ISBN, pages, available, dueDate, fineRate.

Differentiation:

Bronze: Given type, fill in declaration; match variable to type

Silver: Choose appropriate types for 10 data items; explain choices

Gold: Discuss memory implications of type choices; when to use INTEGER vs REAL; precision issues

Lesson 2 (40 min): Input, Output & Expressions 输入、输出与表达式

Learning Objectives:

- ☐ Write pseudocode for input and output operations 编写输入和输出操作的伪代码
- ☐ Construct arithmetic and logical expressions 构建算术和逻辑表达式
- ☐ Apply operator precedence 应用运算符优先级

Starter (5 min): What is $3 + 4 \times 5$? Is it 35 or 23? Why? Introduce precedence.

Main Activity (20 min):

- ☐ Input: INPUT Name — prompts user, stores response in variable.
- ☐ Output: OUTPUT "Hello, ", Name — displays text and variables.
- ☐ Concatenation: OUTPUT "Your score is: ", Score
- ☐ Arithmetic operators: +, -, *, /, DIV (integer division), MOD (remainder)
 - $17 \text{ DIV } 5 = 3$ (how many 5s in 17)
 - $17 \text{ MOD } 5 = 2$ (what's left over)
 - Practical use: MOD for checking even/odd, DIV for splitting digits.
- ☐ Logical operators: AND, OR, NOT
 - $\text{TRUE AND TRUE} = \text{TRUE}$
 - $\text{TRUE AND FALSE} = \text{FALSE}$
 - $\text{TRUE OR FALSE} = \text{TRUE}$
 - $\text{NOT TRUE} = \text{FALSE}$
- ☐ Comparison operators: =, <=, >, <, >=, <=
- ☐ Operator precedence (highest to lowest):

1. Parentheses ()
2. NOT
3. *, /, DIV, MOD, AND
4. +, -, OR
5. Comparison operators

- ☐ Example: $3 + 4 * 5 = 23$ (not 35). $(3 + 4) * 5 = 35$.

Activity (10 min): Calculate expressions with precedence. Identify correct answers.

Plenary (5 min): "Why does MOD help determine if a year is a leap year?" $\text{Year MOD } 4 = 0$ and additional rules.

Homework: Write pseudocode to input 3 numbers, calculate average, output result with message.

Bilingual Keywords:

- ☐ Input 输入 (shūrù)

- ☐ Output 输出 (shūchū)
- ☐ Expression 表达式 (biǎodá shì)
- ☐ Precedence 优先级 (yōuxiān jí)
- ☐ Concatenation 连接 (liánjiē)

Lesson 3 (40 min): Programming Practice & Baseline Test 编程练习与基线测试

Learning Objectives:

- ☐ Combine variables, I/O, and expressions in complete programs 在完整程序中组合变量、输入输出和表达式
- ☐ Write pseudocode to solve simple problems 编写伪代码解决简单问题
- ☐ Trace simple programs by hand 手工追踪简单程序

Starter (5 min): "Predict the output" — Given pseudocode with 3 lines, what displays?

Main Activity (20 min):

- ☐ Worked example: Rectangle area calculator

DECLARE Length : REAL

DECLARE Width : REAL

DECLARE Area : REAL

OUTPUT "Enter length:"

INPUT Length

OUTPUT "Enter width:"

INPUT Width

Area ← Length * Width

OUTPUT "Area = ", Area

- ☐ Students write: Circle circumference and area (using CONSTANT Pi)
- ☐ Students write: Currency converter (input GBP, output CNY at fixed rate)
- ☐ Trace table practice: Track variable values line by line.

Activity (10 min): Peer review — Swap pseudocode, partner traces and verifies.

Plenary (5 min): Common errors from class work: Missing declarations, wrong arrows, type mismatches.

Week 1 Assessment: Baseline Programming Test (30 minutes)

- ☐ Q1: Declare 5 variables for a given scenario (5 marks)
- ☐ Q2: Write pseudocode to calculate and output BMI from height and weight (8 marks)
- ☐ Q3: Trace table for given 5-line program (7 marks)
- ☐ Q4: Evaluate expressions with precedence (5 marks)
- ☐ Q5: Identify errors in given pseudocode (5 marks)
- ☐ Total: 30 marks — Formative, diagnostic grouping

Week 2: Selection & Iteration 选择与循环

Lesson 1 (40 min): Selection — IF, ELSE, Nested IF, CASE 选择结构: IF、ELSE、嵌套IF、CASE

Learning Objectives:

- ☐ Write pseudocode using IF...THEN...ELSE 使用IF...THEN...ELSE编写伪代码
- ☐ Use nested IF for multiple conditions 使用嵌套IF处理多个条件
- ☐ Apply CASE for multiple discrete options 应用CASE处理多个离散选项

Starter (5 min): "Should I bring an umbrella?" Flowchart decision: Is it raining? Is it cloudy? Will it rain?

Main Activity (20 min):

☐ IF...THEN: Single condition, single action.

IF Score >= 50 THEN

OUTPUT "Pass"

ENDIF

☐ IF...THEN...ELSE: Two paths.

IF Score >= 50 THEN

OUTPUT "Pass"

ELSE

OUTPUT "Fail"

ENDIF

☐ Nested IF: Multiple conditions.

IF Score >= 80 THEN

OUTPUT "A"

ELSE

IF Score >= 70 THEN

OUTPUT "B"

ELSE

IF Score >= 60 THEN

OUTPUT "C"

ELSE

OUTPUT "D"

ENDIF

ENDIF

ENDIF

☐ CASE: Cleaner for multiple discrete values.

CASE Grade OF

"A" : OUTPUT "Excellent"

"B" : OUTPUT "Good"

"C" : OUTPUT "Satisfactory"

"D" : OUTPUT "Pass"

OTHERWISE OUTPUT "Fail"

ENDCASE

☐ Comparison: When to use IF vs CASE? CASE for discrete values (menu choices, grades, days). IF for ranges and complex conditions.

☐ Common error: Missing ENDIF. Every IF needs matching ENDIF. CIE is strict about this.

Activity (10 min): Write pseudocode for: "Input month number (1-12), output season." First with nested IF, then with CASE. Compare readability.

Plenary (5 min): "Why is CASE better than 12 nested IFs for menu selection?" Discuss readability, maintainability, efficiency.

Homework: Write pseudocode for a simple calculator: input two numbers and operator (+, -, *, /), output result. Use CASE for operator selection.

Common Misconceptions:

- ☐ ☐ "ELSE is optional" — Technically yes, but omitting it when needed causes logic errors
- ☐ ☐ "You can use CASE with ranges like CASE Score OF 0–59" — CIE CASE uses discrete values, not ranges. Use IF for ranges.

Lesson 2 (40 min): Iteration — FOR, WHILE, REPEAT 循环结构: FOR、WHILE、REPEAT

Learning Objectives:

- ☐ Write programs using FOR loops 使用FOR循环编写程序
- ☐ Write programs using WHILE loops 使用WHILE循环编写程序
- ☐ Write programs using REPEAT...UNTIL loops 使用REPEAT...UNTIL循环编写程序
- ☐ Select appropriate loop type for given problem 为给定问题选择合适的循环类型

Starter (5 min): "I want to print numbers 1 to 10." Could use 10 OUTPUT statements. Better way? Introduce loops.

Main Activity (20 min):

- ☐ FOR loop: Definite iteration (known number of repetitions).

FOR Count = 1 TO 10

OUTPUT Count

ENDFOR

— Can count down: FOR Count = 10 TO 1 STEP -1

— Can step by other amounts: FOR Count = 0 TO 100 STEP 5

- ☐ WHILE loop: Indefinite iteration (condition checked BEFORE each iteration).

DECLARE Password : STRING

Password ← ""

WHILE Password <> "secret" DO

OUTPUT "Enter password:"

INPUT Password

ENDWHILE

OUTPUT "Access granted"

— May execute zero times if condition initially false.

- ☐ REPEAT...UNTIL loop: Indefinite iteration (condition checked AFTER each iteration).

DECLARE Choice : STRING

REPEAT

OUTPUT "Menu: A-Add, D-Delete, Q-Quit"

INPUT Choice

UNTIL Choice = "Q"

— Always executes at least once.

- ☐ Comparison table:

Loop	When to Use	Check	Min Executions
FOR	Known count	Automatic	0 (if start > end)
WHILE	Unknown count, may be zero	Before	0

Loop	When to Use	Check	Min Executions
REPEAT	Unknown count, at least once	After	1

Activity (10 min): For each scenario, choose loop type and write pseudocode: (a) Print 5 times table, (b) Guess number game, (c) Input valid date.

Plenary (5 min): "What happens if you forget to update the condition in a WHILE loop?" Infinite loop! Demo with WHILE TRUE DO...

Homework: Write 3 programs: (1) FOR loop — sum of first 100 integers, (2) WHILE loop — keep doubling until > 1000, (3) REPEAT loop — menu system.

Bilingual Keywords:

- ☐ Loop 循环 (xúnhuán)
- ☐ Iteration 迭代 (diédài)
- ☐ Condition 条件 (tiáojiàn)
- ☐ Infinite loop 无限循环 (wúxiàn xúnhuán)

Lesson 3 (40 min): Nested Loops & Complex Patterns 嵌套循环与复杂模式

Learning Objectives:

- ☐ Write nested loop structures 编写嵌套循环结构
- ☐ Use loops to process patterns and tables 使用循环处理模式和表格
- ☐ Trace nested loops with trace tables 用追踪表追踪嵌套循环

Starter (5 min): How would you print a 5×5 grid of asterisks? One loop for rows, one for columns.

Main Activity (20 min):

- ☐ Nested FOR loops: Outer loop controls rows, inner loop controls columns.

FOR Row = 1 TO 5

FOR Col = 1 TO 5

OUTPUT "*" // Without newline

ENDFOR

OUTPUT "" // Newline

ENDFOR

- ☐ Triangle patterns:

FOR Row = 1 TO 5

FOR Col = 1 TO Row

OUTPUT "*"

ENDFOR

OUTPUT ""

ENDFOR

- ☐ Times table grid:

FOR i = 1 TO 12

FOR j = 1 TO 12

OUTPUT i * j, " "

ENDFOR

OUTPUT ""

ENDFOR

- ☐ Trace table for nested loops: Track outer loop variable, inner loop variable, output.
- ☐ Common error: Forgetting to reset inner loop variables or using same variable name for both loops.

Activity (10 min): Write pseudocode for multiplication table (1-12). Trace for first 3 rows.

Plenary (5 min): "How many times does the inner loop body execute in a 10×10 nested loop?" 100 times. Time complexity $O(n^2)$.

Homework: Write pseudocode to print: (a) Hollow square, (b) Number pyramid, (c) Chessboard pattern (alternating X and 0).

Differentiation:

Bronze: Complete trace table for given nested loop; identify pattern

Silver: Write nested loop for simple pattern (solid rectangle)

Gold: Write nested loop for complex pattern (diamond, Pascal's triangle) with efficient logic

Week 3: Nested Structures, Compound Conditions & Flags 嵌套结构、复合条件与标志

Lesson 1 (40 min): Compound Conditions & Logical Operators 复合条件与逻辑运算符

Learning Objectives:

- ☐ Combine conditions using AND, OR, NOT 使用AND、OR、NOT组合条件
- ☐ Construct truth tables for complex conditions 为复杂条件构建真值表
- ☐ Apply De Morgan's laws 应用德摩根定律

Starter (5 min): "You can go to the party if you finish homework AND clean your room." Is going with just homework done? No. Need BOTH.

Main Activity (20 min):

- ☐ Compound conditions: Multiple conditions combined.
- ☐ AND: Both must be true. IF Age $\geq$ 18 AND HasLicense = TRUE THEN
- ☐ OR: At least one must be true. IF Day = "Saturday" OR Day = "Sunday" THEN
- ☐ NOT: Reverses condition. IF NOT IsMember THEN (same as IF IsMember = FALSE THEN)
- ☐ Truth tables for AND, OR, NOT:

A	B	A AND B	A OR B	NOT A
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

☐ De Morgan's Laws:

— NOT (A AND B) = (NOT A) OR (NOT B)

— NOT (A OR B) = (NOT A) AND (NOT B)

— Example: NOT (Age $\geq$ 18 AND HasLicense) = Age $<$ 18 OR NOT HasLicense

☐ Short-circuit evaluation: In A AND B, if A is false, B is not evaluated. In A OR B, if A is true, B is not evaluated.

Activity (10 min): Simplify conditions using De Morgan's laws. Evaluate compound conditions.

Plenary (5 min): "Why is short-circuit evaluation useful?" Discuss: efficiency, preventing errors (e.g., $x \geq 0$ AND $(10/x \geq 2)$).

Homework: Create truth table for (A AND B) OR (NOT C) with 8 rows. Write pseudocode using compound conditions for login validation (username AND password AND NOT locked).

Common Misconceptions:

- ☐ ☐ "A AND B means either A or B is true" — No, BOTH must be true
- ☐ ☐ "NOT (A AND B) = NOT A AND NOT B" — Wrong! It's NOT A OR NOT B (De Morgan's)

Lesson 2 (40 min): Flag Variables & State Tracking 标志变量与状态跟踪

Learning Objectives:

- ☐ Use Boolean flags to control program flow 使用布尔标志控制程序流程
- ☐ Implement search algorithms with flags 使用标志实现搜索算法
- ☐ Design state-based solutions 设计基于状态的解决方案

Starter (5 min): "Find the word 'apple' in a list of 100 fruits." How do you know if you found it or not? Introduce flag.

Main Activity (20 min):

- ☐ Flag variable: Boolean variable tracking whether something has occurred. 标志变量：跟踪某事是否发生的布尔变量。
- ☐ Search example:

```
DECLARE Found : BOOLEAN
```

```
Found ← FALSE
```

```
FOR i = 1 TO 100
```

```
  IF Fruits[i] = "Apple" THEN
```

```
    Found ← TRUE
```

```
  ENDIF
```

```
ENDFOR
```

```
IF Found = TRUE THEN
```

```
  OUTPUT "Apple is in the list"
```

```
ELSE
```

```
  OUTPUT "Apple not found"
```

```
ENDIF
```

- ☐ Optimised search (exit early when found):

```
Found ← FALSE
```

```
i ← 1
```

```
WHILE i <= 100 AND Found = FALSE DO
```

```
  IF Fruits[i] = "Apple" THEN
```

```
    Found ← TRUE
```

```
  ELSE
```

```
    i ← i + 1
```

```
  ENDIF
```

```
ENDWHILE
```

- ☐ State tracking: Using flags for multi-stage processes.

— Example: Password validation — needs uppercase, lowercase, digit, special char.

— Four flags, all must be true for valid password.

Activity (10 min): Write pseudocode to check if a number is prime using flag. Test with 17 and 18.

Plenary (5 min): "When is a flag better than a direct condition?" Discuss: multiple checks, early exit, complex state tracking.

Homework: Write pseudocode for password validator with 4 flags (upper, lower, digit, length).

Bilingual Keywords:

- ☐ Flag 标志 (biāozhì)
- ☐ State 状态 (zhuàngtài)
- ☐ Search 搜索 (sōusuǒ)
- ☐ Valid 有效 (yǒuxiào)

Lesson 3 (40 min): Problem Solving with Pseudocode 用伪代码解决问题

Learning Objectives:

- ☐ Apply stepwise refinement to problems 对问题应用逐步求精
- ☐ Write complete pseudocode solutions 编写完整的伪代码解决方案
- ☐ Trace and debug pseudocode 追踪和调试伪代码

Starter (5 min): Problem: "Calculate total price including 8% tax and 5% discount for VIP." Break it down.

Main Activity (20 min):

- ☐ Stepwise refinement:

Level 1: Input → Calculate → Output

Level 2: Get price, get VIP status → Calculate discount → Calculate tax → Calculate total → Display result

Level 3: Pseudocode implementation

- ☐ Complete example:

```
DECLARE Price, DiscountRate, TaxRate, Total : REAL
```

```
DECLARE IsVIP : BOOLEAN
```

```
Price ← 0
```

```
DiscountRate ← 0.05
```

```
TaxRate ← 0.08
```

```
OUTPUT "Enter price:"
```

```
INPUT Price
```

```
OUTPUT "Is VIP? (TRUE/FALSE):"
```

```
INPUT IsVIP
```

```
IF IsVIP = TRUE THEN
```

```
Price ← Price * (1 - DiscountRate)
```

```
ENDIF
```

```
Total ← Price * (1 + TaxRate)
```

```
OUTPUT "Total: $", Total
```

- ☐ Debugging exercise: Given buggy code with 5 errors, identify and fix.
- ☐ Common bugs: Missing ENDIF, wrong comparison operator, forgetting to initialise, infinite loops, off-by-one errors.

Activity (10 min): "Bug hunt" — 5 programs with deliberate errors. Find and fix.

Plenary (5 min): Debugging checklist: (1) Declarations present? (2) Every IF has ENDIF? (3) Loop condition updates? (4) Correct data types?

Homework: Write pseudocode for ATM withdrawal: check balance, check multiple of 10, update balance, output receipt info.

Week 3 Assessment: Code Tracing Quiz (20 minutes)

- ☐ Q1: Trace table for IF...ELSE with compound condition (5 marks)
- ☐ Q2: Trace table for nested FOR loops (5 marks)
- ☐ Q3: Identify 3 errors in given pseudocode (6 marks)
- ☐ Q4: Write pseudocode for given specification (9 marks)
- ☐ Total: 25 marks — Formative

Week 4: Subroutines — Procedures & Functions 子程序：过程与函数

Lesson 1 (40 min): Procedures 过程

Learning Objectives:

- ☐ Explain purpose of procedures 解释过程的目的
- ☐ Write pseudocode for procedures with parameters 编写带参数的过程伪代码
- ☐ Understand scope of variables 理解变量的作用域

Starter (5 min): You need to greet 10 different students. Without copy-pasting code 10 times, how?

Main Activity (20 min):

- ☐ Procedure: Named block of code that performs a task. No return value. 过程：执行任务的命名代码块，无返回值。
- ☐ Syntax:

```
PROCEDURE Greet(Name : STRING)
```

```
OUTPUT "Hello, ", Name, "!"
```

```
ENDPROCEDURE
```

```
// Main program
```

```
CALL Greet("Alice")
```

```
CALL Greet("Bob")
```

- ☐ Parameters: Values passed to procedure. 参数：传递给过程的值。

- ☐ Multiple parameters:

```
PROCEDURE DisplayResult(Subject : STRING, Score : INTEGER, Grade : STRING)
```

```
OUTPUT Subject, ": ", Score, " (" , Grade, ")"
```

```
ENDPROCEDURE
```

- ☐ Scope: Variables declared inside procedure are LOCAL (only accessible there). 作用域：过程内声明的变量是局部的。
- ☐ Global variables: Declared outside, accessible everywhere. 全局变量：外部声明，到处可访问。
- ☐ Best practice: Minimise global variables. Use parameters and local variables. 最佳实践：最小化全局变量。

Activity (10 min): Write procedure to print a rectangle of asterisks given width and height. Call it 3 times with different sizes.

Plenary (5 min): "Why are global variables considered bad practice?" Discuss: unintended side effects, hard to debug, not reusable.

Homework: Write 3 procedures for a calculator: Add, Subtract, Multiply. Each takes 2 parameters and outputs result.

Bilingual Keywords:

- ☐ Procedure 过程 (guòchéng)
- ☐ Parameter 参数 (cānshù)

- ☐ Argument 实参 (shí cān)
- ☐ Scope 作用域 (zuò yòng yù)
- ☐ Local 局部的 (jú bù de)
- ☐ Global 全局的 (quán jú de)

Lesson 2 (40 min): Functions 函数

Learning Objectives:

- ☐ Distinguish functions from procedures 区分函数与过程
- ☐ Write functions that return values 编写返回值的函数
- ☐ Use functions in expressions 在表达式中使用函数

Starter (5 min): In maths, $f(x) = 2x + 1$. What is $f(3)$? Functions in programming are similar — they RETURN a value.

Main Activity (20 min):

- ☐ Function: Subroutine that returns a value to the calling code. 函数: 向调用代码返回值的子程序。
- ☐ Syntax:

```
FUNCTION Max(a : INTEGER, b : INTEGER) RETURNS INTEGER
```

```
  IF a > b THEN
```

```
    RETURN a
```

```
  ELSE
```

```
    RETURN b
```

```
  ENDIF
```

```
ENDFUNCTION
```

```
// Usage
```

```
Larger ← Max(5, 3) // Larger = 5
```

```
OUTPUT Max(10, 20) // Outputs 20
```

- ☐ Key difference from procedure: Function has RETURN statement and RETURNS type.
关键区别: 函数有RETURN语句和返回类型。

- ☐ Functions can be used in expressions: $\text{Result} \leftarrow \text{Max}(x, y) + \text{Max}(a, b)$

- ☐ Common function examples:

```
— FUNCTION IsEven(n : INTEGER) RETURNS BOOLEAN
```

```
— FUNCTION CircleArea(radius : REAL) RETURNS REAL
```

```
— FUNCTION GradeFromScore(Score : INTEGER) RETURNS STRING
```

- ☐ Functions should be PURE when possible: Same input always gives same output. No side effects.
纯函数: 相同输入总是产生相同输出。

Activity (10 min): Write functions: (a) Fahrenheit to Celsius, (b) IsLeapYear, (c) Factorial (iterative).

Plenary (5 min): "Can a function contain OUTPUT statements?" Discuss: technically yes, but functions should calculate and return, not display. Mixed responsibilities.

Homework: Write function library: AreaOfRectangle, AreaOfTriangle, AreaOfCircle. Main program asks user which shape, calls appropriate function.

Common Misconceptions:

- ☐ ☐ "Functions and procedures are the same" — Functions return values; procedures don't (or shouldn't)
- ☐ ☐ "You can only RETURN at the end" — RETURN can appear anywhere, but single exit point is cleaner

Lesson 3 (40 min): Parameter Passing & Practical Test 参数传递与实践测试

Learning Objectives:

- ☐ Explain pass-by-value and pass-by-reference 解释值传递和引用传递
- ☐ Analyse effects of parameter passing on variables 分析参数传递对变量的影响
- ☐ Complete practical programming test 完成实践编程测试

Starter (5 min): You give a friend a photocopy of your essay (value) vs the original document (reference). What's the difference if they change it?

Main Activity (20 min):

- ☐ Pass by value: Copy of data sent to subroutine. Original unchanged. 值传递: 发送数据的副本, 原始数据不变。
- ☐ Pass by reference: Address sent. Subroutine can modify original. 引用传递: 发送地址, 子程序可以修改原始数据。
- ☐ CIE pseudocode: By default, parameters are pass by value. To indicate pass by reference, some systems use BYREF keyword (context-dependent in CIE).
- ☐ Example demonstrating difference:

```
PROCEDURE Triple(x : INTEGER)
```

```
  x ← x * 3
```

```
OUTPUT "Inside: ", x
```

```
ENDPROCEDURE
```

```
// Main
```

```
Num ← 5
```

```
CALL Triple(Num)
```

```
OUTPUT "Outside: ", Num // Num is still 5 (pass by value)
```

- ☐ If pass by reference, Num would become 15.
- ☐ Implications: Use pass by value for input data. Use pass by reference when subroutine needs to modify caller's data.

Activity (10 min): Predict output for 3 parameter-passing scenarios. Test understanding.

Plenary (5 min): "Why is pass by reference more efficient for large arrays?" Discuss: avoids copying entire array.

Week 4 Assessment: Practical Programming Test (45 minutes)

- ☐ Task 1: Write procedure to display a menu (6 marks)
- ☐ Task 2: Write function to calculate BMI from weight and height (8 marks)
- ☐ Task 3: Write main program using both, with input validation (10 marks)
- ☐ Task 4: Trace a given subroutine call with parameters (6 marks)
- ☐ Total: 30 marks — 10% of semester grade
- ☐ 实践测试: 编写完整程序, 展示对子程序的理解。

Week 5: Recursion 递归 (Dìguī)

Lesson 1 (40 min): Introduction to Recursion 递归入门

Learning Objectives:

- ☐ Define recursion and identify recursive patterns 定义递归并识别递归模式
- ☐ Explain base case and recursive case 解释基准情况和递归情况
- ☐ Trace simple recursive functions 追踪简单递归函数

Starter (5 min): Russian dolls — each doll contains a smaller doll. When does it stop? Smallest doll = base case.

Main Activity (20 min):

☐ Recursion: Function calls itself to solve smaller instance of same problem. 递归: 函数调用自身解决同类更小问题。

☐ Two essential components:

1. Base case: Condition that stops recursion. 基准情况: 停止递归的条件。

2. Recursive case: Function calls itself with simpler input. 递归情况: 函数用更简单的输入调用自身。

☐ Factorial example:

FUNCTION Factorial(n : INTEGER) RETURNS INTEGER

IF n <= 1 THEN

RETURN 1

ELSE

RETURN n * Factorial(n - 1)

ENDIF

ENDFUNCTION

Factorial(4) = 4 × Factorial(3) = 4 × 3 × Factorial(2) = 4 × 3 × 2 × Factorial(1) = 4 × 3 × 2 × 1 = 24

☐ Stack frames: Each recursive call adds frame to call stack. When base case reached, frames unwind. 栈帧: 每次递归调用向调用栈添加一帧。

☐ Memory risk: Deep recursion causes stack overflow. 内存风险: 深层递归导致栈溢出。

Activity (10 min): Trace Factorial(5) on paper. Show stack frames at each level.

Plenary (5 min): "What happens if you forget the base case?" Infinite recursion → stack overflow → crash.

Homework: Write recursive function to calculate sum of first n integers. Trace for n=4.

Common Misconceptions:

☐ ☐ "Recursion is just a loop" — Different mechanism (stack vs iteration), different use cases

☐ ☐ "Recursion is always better" — Iteration often more efficient (no stack overhead)

Lesson 2 (40 min): Recursive Algorithms — Fibonacci & Search 递归算法: 斐波那契与搜索

Learning Objectives:

☐ Implement Fibonacci recursively 递归实现斐波那契

☐ Compare recursive and iterative solutions 比较递归和迭代解决方案

☐ Understand when recursion is appropriate 理解何时使用递归

Starter (5 min): Fibonacci sequence: 1, 1, 2, 3, 5, 8, 13... Each number = sum of two previous. How to code this?

Main Activity (20 min):

☐ Fibonacci recursive:

FUNCTION Fibonacci(n : INTEGER) RETURNS INTEGER

IF n <= 2 THEN

RETURN 1

ELSE

RETURN Fibonacci(n - 1) + Fibonacci(n - 2)

ENDIF

ENDFUNCTION

☐ Trace Fibonacci(4):

Fib(4) = Fib(3) + Fib(2)

Fib(3) = Fib(2) + Fib(1) = 1 + 1 = 2

Fib(2) = 1

Fib(4) = 2 + 1 = 3

☐ Problem: Recursive Fibonacci is very inefficient! Fib(5) calls Fib(3) twice. Exponential time $O(2^n)$.

☐ Iterative Fibonacci: $O(n)$ time, $O(1)$ space. Much better for large n .

☐ When to use recursion:

— Tree/graph traversal (natural fit)

— Divide and conquer (merge sort, quick sort)

— Problems with recursive structure (file systems, nested menus)

— When code clarity matters more than performance

☐ When to use iteration:

— Simple sequential processing

— Performance-critical code

— Very deep processing (risk of stack overflow)

Activity (10 min): Compare recursive vs iterative for summing array. Which is faster? Why?

Plenary (5 min): "Recursion is elegant but dangerous." Discuss: readability vs stack limits, efficiency.

Homework: Write recursive function for power(base, exponent). Trace power(2, 5). Compare with iterative version.

Bilingual Keywords:

☐ Recursion 递归 (dìguī)

☐ Base case 基准情况 (jīzhǔn qíngkuàng)

☐ Recursive case 递归情况 (dìguī qíngkuàng)

☐ Stack overflow 栈溢出 (zhàn yìchū)

☐ Call stack 调用栈 (diàoyòng zhàn)

Lesson 3 (40 min): Recursion Worksheet & Review 递归练习与复习

Learning Objectives:

☐ Trace complex recursive programs 追踪复杂递归程序

☐ Identify base and recursive cases in given code 识别给定代码中的基准和递归情况

☐ Convert recursive solutions to iterative 将递归解决方案转换为迭代

Starter (5 min): Quick trace: What does this output?

FUNCTION Mystery(n : INTEGER) RETURNS INTEGER

IF $n = 0$ THEN

RETURN 0

ELSE

RETURN $n + \text{Mystery}(n - 1)$

ENDIF

ENDFUNCTION

OUTPUT Mystery(5)

Main Activity (20 min):

☐ Worksheet with 5 recursive functions. For each:

— Identify base case

— Identify recursive case

— Trace for given input

— State what the function calculates

☐ Functions include: Countdown, Digit sum, String reverse, Power, Count characters.

☐ Conversion exercise: Convert recursive countdown to iterative FOR loop.

☐ Exam-style question: "Explain why the recursive solution to Factorial is less efficient than an iterative solution for large values of n." (3 marks)

Activity (10 min): Challenge: Write recursive function to check if string is palindrome.

Plenary (5 min): Review worksheet answers. Address common tracing errors.

Week 5 Assessment: Recursion Worksheet (30 minutes)

☐ Q1: Trace recursive function (factorial variant) — 6 marks

☐ Q2: Identify base case and recursive case in 3 functions — 6 marks

☐ Q3: Convert recursive sum function to iterative — 5 marks

☐ Q4: Explain why recursion uses more memory than iteration — 3 marks

☐ Q5: Write recursive function for given specification — 5 marks

☐ Total: 25 marks — Formative

Week 6: String & File Handling 字符串与文件处理

Lesson 1 (40 min): String Operations 字符串操作

Learning Objectives:

☐ Use string functions in pseudocode 在伪代码中使用字符串函数

☐ Manipulate strings using substring and concatenation 使用子串和连接操作字符串

☐ Convert between strings and other types 在字符串和其他类型之间转换

Starter (5 min): How many characters in "Hello, 世界"? How to extract first 3 characters? Last 2?

Main Activity (20 min):

☐ CIE string functions:

— LENGTH(Text) → number of characters

— Text[Start:End] → substring from position Start to End inclusive

— LEFT(Text, n) → first n characters

— RIGHT(Text, n) → last n characters

— INT(String) → converts string to integer

— STR(Number) → converts number to string

— ASC(Character) → ASCII code

— CHR(Code) → character from ASCII code

☐ Examples:

LENGTH("Hello") = 5

"Hello"[1:3] = "Hel"

STR(42) = "42"

INT("123") = 123

ASC("A") = 65

CHR(65) = "A"

☐ String traversal: Loop through each character.

```
DECLARE Text : STRING
```

```
DECLARE i : INTEGER
```

```
Text ← "Hello"
```

```
FOR i = 1 TO LENGTH(Text)
```

```
OUTPUT Text[i:i]
```

```
ENDFOR
```

☐ Practical applications: Password validation, text formatting, data parsing.

Activity (10 min): Write pseudocode to: (a) Reverse a string, (b) Count vowels, (c) Check if palindrome.

Plenary (5 min): "Why is string indexing 1-based in CIE pseudocode?" Discuss: different from Python (0-based), exam-specific convention.

Homework: Write pseudocode to validate email format: contains @, contains . after @, at least 5 characters total.

Bilingual Keywords:

☐ String 字符串 (zìfúchuàn)

☐ Substring 子串 (zǐchuàn)

☐ Concatenation 连接 (liánjiē)

☐ Length 长度 (chángdù)

☐ ASCII ASCII码

Lesson 2 (40 min): File Handling — Read & Write 文件处理: 读写

Learning Objectives:

☐ Open, read, write, and close files in pseudocode 在伪代码中打开、读取、写入和关闭文件

☐ Process sequential files 处理顺序文件

☐ Handle end-of-file conditions 处理文件结束条件

Starter (5 min): Where does your game save progress? In a file! How does the program read it back?

Main Activity (20 min):

☐ File operations in CIE pseudocode:

```
— OPENFILE "data.txt" FOR READ
```

```
— OPENFILE "output.txt" FOR WRITE (creates new/overwrite)
```

```
— OPENFILE "log.txt" FOR APPEND (adds to end)
```

```
— READFILE "data.txt", Line
```

```
— WRITEFILE "output.txt", "Hello"
```

```
— CLOSEFILE "data.txt"
```

☐ Reading until end of file:

```
OPENFILE "scores.txt" FOR READ
```

```
WHILE NOT EOF("scores.txt") DO
```

```
READFILE "scores.txt", Score
```

```
OUTPUT Score
```

```
ENDWHILE
```

```
CLOSEFILE "scores.txt"
```

☐ Writing to file:

```

OPENFILE "results.txt" FOR WRITE
WRITEFILE "results.txt", "Name, Score, Grade"
FOR i = 1 TO 10
WRITEFILE "results.txt", Names[i] & ", " & STR(Scores[i]) & ", " & Grades[i]
ENDFOR
CLOSEFILE "results.txt"

```

☐ Important: Always close files! Open files consume resources. 重要: 始终关闭文件!

☐ Error handling: Check file exists before reading. Handle empty files gracefully.

Activity (10 min): Write pseudocode to read 5 names from file, output them with line numbers.

Plenary (5 min): "What's the difference between FOR WRITE and FOR APPEND?" Discuss: overwrite vs preserve.

Homework: Write pseudocode to merge two sorted files into one sorted file.

Common Misconceptions:

☐ ☐ "READFILE reads entire file at once" — It reads one line/record per call

☐ ☐ "You don't need to close files" — Resource leak, data may not be saved

Lesson 3 (40 min): CSV Processing CSV处理

Learning Objectives:

☐ Parse CSV data using string functions 使用字符串函数解析CSV数据

☐ Construct CSV records from data 从数据构建CSV记录

☐ Process structured data files 处理结构化数据文件

Starter (5 min): Show real CSV: Name, Age, Score

Alice,16,85

Bob,17,72. How to extract Alice's score?

Main Activity (20 min):

☐ CSV (Comma Separated Values): Plain text format for tabular data. CSV: 表格数据的纯文本格式。

☐ Parsing CSV: Split line by comma delimiter.

// Given: Line = "Alice,16,85"

DECLARE Name : STRING

DECLARE Age : INTEGER

DECLARE Score : INTEGER

DECLARE Comma1, Comma2 : INTEGER

Comma1 ← 0

Comma2 ← 0

// Find first comma

FOR i = 1 TO LENGTH(Line)

IF Line[i:i] = "," AND Comma1 = 0 THEN

Comma1 ← i

ELSE

IF Line[i:i] = "," AND Comma1 <> 0 THEN

Comma2 ← i

ENDIF

ENDIF

ENDFOR

Name ← Line[1:Comma1-1]

Age ← INT(Line[Comma1+1:Comma2-1])

Score ← INT(Line[Comma2+1:LENGTH(Line)])

☐ CIE simplified approach (using built-in where available):

DECLARE Fields : ARRAY[1:3] OF STRING

// Conceptually split by comma

☐ Writing CSV: Ensure data separated by commas, strings may need quotes if containing commas.

☐ Practical: Read student data from CSV, calculate average, write results to new CSV.

Activity (10 min): Given CSV lines, write pseudocode to find highest score and output name.

Plenary (5 min): "Why is CSV still popular despite XML and JSON?" Discuss: simplicity, human-readable, universal compatibility.

Week 6 Assessment: Mid-Unit Test — Programming Constructs (45 minutes)

☐ Q1: Write pseudocode to reverse a string using loop (5 marks)

☐ Q2: Read numbers from file until EOF, count how many are positive (6 marks)

☐ Q3: Write pseudocode to parse CSV line and extract fields (6 marks)

☐ Q4: Write function to check if password is valid (8 marks)

☐ Q5: Trace recursive function with given input (5 marks)

☐ Q6: Explain difference between function and procedure (3 marks)

☐ Q7: Write pseudocode for menu-driven program using subroutines (7 marks)

☐ Total: 40 marks — 10% of semester grade

Week 7: 1D Arrays — Declaration, Traversal, Search, Sort 一维数组：声明、遍历、搜索、排序

Lesson 1 (40 min): Array Basics & Traversal 数组基础与遍历

Learning Objectives:

☐ Declare and initialise 1D arrays 声明和初始化一维数组

☐ Traverse arrays using loops 使用循环遍历数组

☐ Access and modify array elements 访问和修改数组元素

Starter (5 min): 30 students in class, each has a test score. 30 variables? No! Use array.

Main Activity (20 min):

☐ Array: Collection of elements of same type, accessed by index. 数组：同类型元素的集合，通过索引访问。

☐ Declaration: DECLARE Scores : ARRAY[1:30] OF INTEGER

☐ Access: Scores[5] → element at position 5.

☐ Initialisation: Loop to set default values.

DECLARE Scores : ARRAY[1:30] OF INTEGER

DECLARE i : INTEGER

FOR i = 1 TO 30

Scores[i] ← 0

ENDFOR

☐ Traversal: Visit every element.

FOR i = 1 TO 30

OUTPUT "Student ", i, ": ", Scores[i]

ENDFOR

☐ Common error: Off-by-one. Accessing Scores[0] or Scores[31] when array is [1:30]. Index out of bounds.
常见错误: 越界访问。

☐ Parallel arrays: Multiple arrays with same index representing related data.

Names[1] and Scores[1] belong to same student.

Activity (10 min): Declare array for 7 days temperature. Input values, calculate average.

Plenary (5 min): "Why do arrays start at index 1 in CIE pseudocode?" Discuss: exam convention, different from some languages.

Homework: Write pseudocode to find maximum and minimum in array of 50 numbers.

Bilingual Keywords:

☐ Array 数组 (shùzǔ)

☐ Index 索引 (suǒyǐn)

☐ Element 元素 (yuánsù)

☐ Traversal 遍历 (biànlì)

☐ Bounds 边界 (biānjiè)

Lesson 2 (40 min): Linear Search & Bubble Sort 线性搜索与冒泡排序

Learning Objectives:

☐ Implement linear search algorithm 实现线性搜索算法

☐ Implement bubble sort algorithm 实现冒泡排序算法

☐ Analyse efficiency of simple algorithms 分析简单算法的效率

Starter (5 min): Find "Zhang" in unordered class list. How? Check each name one by one. Introduce linear search.

Main Activity (20 min):

☐ Linear Search:

FUNCTION LinearSearch(arr : ARRAY, n : INTEGER, target : STRING) RETURNS INTEGER

DECLARE i : INTEGER

FOR i = 1 TO n

IF arr[i] = target THEN

RETURN i

ENDIF

ENDFOR

RETURN -1 // Not found

ENDFUNCTION

— Best case: $O(1)$ — target is first element.

— Worst case: $O(n)$ — target is last or not present.

— Average: $O(n)$.

☐ Bubble Sort:

```
PROCEDURE BubbleSort(arr : ARRAY, n : INTEGER)
```

```
  DECLARE i, j : INTEGER
```

```
  DECLARE temp : INTEGER
```

```
  FOR i = 1 TO n - 1
```

```
    FOR j = 1 TO n - i
```

```
      IF arr[j] > arr[j + 1] THEN
```

```
        temp ← arr[j]
```

```
        arr[j] ← arr[j + 1]
```

```
        arr[j + 1] ← temp
```

```
      ENDIF
```

```
    ENDFOR
```

```
  ENDFOR
```

```
ENDPROCEDURE
```

— Compare adjacent elements, swap if out of order. Largest "bubbles" to end.

— Passes: $n-1$ passes needed.

— Best case: $O(n)$ if already sorted (with optimisation).

— Worst/Average: $O(n^2)$.

— Not suitable for large datasets but easy to understand.

Activity (10 min): Trace bubble sort on [5, 2, 8, 1, 9]. Show array after each pass.

Plenary (5 min): "Why is bubble sort called 'bubble' sort?" Discuss: lighter (smaller) elements bubble up.

Homework: Implement bubble sort with early exit flag (stop if no swaps in a pass). Trace improved version.

Common Misconceptions:

☐ ☐ "Linear search is $O(n)$ always" — Best case is $O(1)$, worst is $O(n)$

☐ ☐ "Bubble sort is the best sorting algorithm" — Very slow for large data; use for education only

Lesson 3 (40 min): Array Practice — Find, Count, Filter 数组练习: 查找、计数、筛选

Learning Objectives:

☐ Write programs to count occurrences in arrays 编写程序计数数组中元素出现次数

☐ Filter arrays based on conditions 根据条件筛选数组

☐ Combine array operations 组合数组操作

Starter (5 min): How many students scored above 80? How many failed (below 50)?

Main Activity (20 min):

☐ Count occurrences:

```
FUNCTION CountAbove(arr : ARRAY, n : INTEGER, threshold : INTEGER) RETURNS INTEGER
```

```
  DECLARE count, i : INTEGER
```

```
  count ← 0
```

```
  FOR i = 1 TO n
```

```
    IF arr[i] > threshold THEN
```

```
      count ← count + 1
```

```
    ENDIF
```

```
  ENDFOR
```

RETURN count

ENDFUNCTION

☐ Filter to new array:

DECLARE Passed : ARRAY[1:50] OF INTEGER

DECLARE PassCount, i : INTEGER

PassCount $\leftarrow$ 0

FOR i = 1 TO 50

IF Scores[i] $\geq$ 50 THEN

PassCount $\leftarrow$ PassCount + 1

Passed[PassCount] $\leftarrow$ Scores[i]

ENDIF

ENDFOR

☐ Combined task: Find average of passed students, count distinctions (≥ 80), output statistics.

Activity (10 min): Write pseudocode for: Given array of 100 integers, output how many are even, how many are odd, how many are prime.

Plenary (5 min): "Why might filtering be inefficient for very large datasets?" Discuss: memory for new array, consider in-place filtering.

Homework: Complete array worksheet: 6 problems covering search, count, filter, average, max/min.

Differentiation:

Bronze: Complete partially written code for counting and filtering

Silver: Write complete pseudocode from specification

Gold: Optimise for single pass through array (calculate multiple stats simultaneously)

Week 8: 2D Arrays & Records 二维数组与记录

Lesson 1 (40 min): 2D Arrays 二维数组

Learning Objectives:

- ☐ Declare and access 2D arrays 声明和访问二维数组
- ☐ Traverse 2D arrays using nested loops 使用嵌套循环遍历二维数组
- ☐ Apply 2D arrays to real problems 将二维数组应用于实际问题

Starter (5 min): Tic-tac-toe grid, spreadsheet, cinema seating. What do they have in common? Rows AND columns.

Main Activity (20 min):

☐ 2D Array: Array of arrays. Two indices: row and column. 二维数组: 数组的数组, 两个索引: 行和列。

☐ Declaration: DECLARE Grid : ARRAY[1:3, 1:3] OF INTEGER

☐ Access: Grid[2, 3] $\rightarrow$ row 2, column 3.

☐ Initialisation with nested loops:

DECLARE Grid : ARRAY[1:3, 1:3] OF INTEGER

DECLARE r, c : INTEGER

FOR r = 1 TO 3

FOR c = 1 TO 3

Grid[r, c] $\leftarrow$ 0

ENDFOR

ENDFOR

- ☐ Row-major traversal: Outer loop = rows, inner = columns.
- ☐ Column-major traversal: Outer loop = columns, inner = rows.
- ☐ Applications: Game boards, matrices, timetables, image pixels, seating plans.
- ☐ Sum of row: FOR c = 1 TO 3: total $\leftarrow$ total + Grid[r, c]
- ☐ Sum of column: FOR r = 1 TO 3: total $\leftarrow$ total + Grid[r, c]

Activity (10 min): Write pseudocode for 3×3 matrix addition. Two arrays input, third array stores sum.

Plenary (5 min): "How is a 2D array stored in memory?" Discuss: Row-major (all of row 1, then row 2...) vs column-major.

Homework: Write pseudocode for cinema booking: 5 rows × 8 seats. Mark booked seats, find available seats.

Bilingual Keywords:

- ☐ 2D array 二维数组 (èr wéi shùzǔ)
- ☐ Row 行 (háng)
- ☐ Column 列 (liè)
- ☐ Matrix 矩阵 (jǔzhèn)

Lesson 2 (40 min): Records (User-Defined Types) 记录 (用户自定义类型)

Learning Objectives:

- ☐ Define record types in pseudocode 在伪代码中定义记录类型
- ☐ Create and access records 创建和访问记录
- ☐ Use arrays of records 使用记录数组

Starter (5 min): Student has name, age, score, grade. Four separate arrays? Better: one structured type.

Main Activity (20 min):

- ☐ Record: Composite data type combining multiple fields. 记录: 组合多个字段的复合数据类型。
- ☐ Type definition:

TYPE Student

Name : STRING

Age : INTEGER

Score : INTEGER

Grade : STRING

ENDTYPE

DECLARE pupil : Student

pupil.Name $\leftarrow$ "Alice"

pupil.Age $\leftarrow$ 16

pupil.Score $\leftarrow$ 85

pupil.Grade $\leftarrow$ "A"

- ☐ Arrays of records:

DECLARE Class : ARRAY[1:30] OF Student

Class[1].Name $\leftarrow$ "Alice"

Class[1].Score $\leftarrow$ 85

☐ Processing array of records:

```
FOR i = 1 TO 30
  IF Class[i].Score >= 80 THEN
    Class[i].Grade ← "A"
  ELSE
    IF Class[i].Score >= 70 THEN
      Class[i].Grade ← "B"
    ENDIF
  ENDIF
ENDFOR
```

☐ Benefits: Logical grouping, cleaner code, easier maintenance.

☐ Comparison with parallel arrays: Records keep related data together. No risk of index mismatch.

Activity (10 min): Define record type for Book (title, author, ISBN, price, available). Create array of 5 books, search by author.

Plenary (5 min): "When would you use records instead of 2D arrays?" Discuss: heterogeneous data, named fields, readability.

Homework: Define record for Car (make, model, year, mileage). Write pseudocode to find oldest car with mileage under 50,000.

Common Misconceptions:

☐ ☐ "Records can contain different types of data in one field" — Each field has fixed type; record contains multiple fields with different types

☐ ☐ "Array of records is the same as 2D array" — No, array of records has named fields; 2D array has same type throughout

Lesson 3 (40 min): Integrated Programming Task 综合编程任务

Learning Objectives:

☐ Combine all programming constructs in complete solution 在完整解决方案中组合所有编程结构

☐ Apply structured design principles 应用结构化设计原则

☐ Write maintainable, readable pseudocode 编写可维护、可读的伪代码

Starter (5 min): Review: What have we learned? Variables, I/O, selection, iteration, subroutines, arrays, records, files. Time to put it together!

Main Activity (25 min):

☐ Task: Student Management System

— Store up to 50 students (record: Name, ID, 3 SubjectScores, Average, Grade)

— Menu-driven: Add student, Display all, Calculate averages, Assign grades, Save to file, Load from file, Exit

— Use subroutines for each menu option

— Use arrays of records

— Validate input

☐ Stepwise refinement:

Level 1: Main loop → Menu → Actions → Exit

Level 2: Each action refined into subroutine

Level 3: Pseudocode implementation

☐ Teacher models: Main program structure and AddStudent subroutine.

☐ Students complete: DisplayAll, CalculateAverages, AssignGrades, SaveToFile, LoadFromFile.

Activity (10 min): Peer review of completed pseudocode. Check: all features, validation, structure.

Plenary (5 min): "What makes code 'good' beyond just working?" Discuss: readability, comments, structure, efficiency, maintainability.

Homework: Complete full pseudocode for Student Management System. Add at least one enhancement (search by name, sort by average).

EAL Support:

- ☐ Provide bilingual flowchart for system design
- ☐ Sentence frames for explaining code: "This subroutine...", "The purpose of this loop is..."
- ☐ Allow Chinese annotations in draft pseudocode

Week 9: Half-Term Assessment & Review 期中评估与复习

Half-Term Examination — Paper 2 Section A Style (Week 9)

Format: 1 hour, 35 marks

Section A: Short Answer (15 marks)

- ☐ Q1: Trace table for selection + iteration (5 marks)
- ☐ Q2: Identify errors in given pseudocode (4 marks)
- ☐ Q3: Explain difference between function and procedure (3 marks)
- ☐ Q4: State output of recursive function trace (3 marks)

Section B: Programming Tasks (20 marks)

- ☐ Q5: Write pseudocode to find and display all prime numbers between 1 and 100 (8 marks)
- ☐ Q6: Write function to calculate average of array and return it (6 marks)
- ☐ Q7: Write pseudocode to read names from file, store in array, output in reverse order (6 marks)

Marking: Teacher-marked with detailed feedback. Focus on CIE pseudocode conventions.

Post-Assessment:

- ☐ W9 L2: Review and error analysis
- ☐ W9 L3: Targeted programming practice for weak areas
- ☐ 考后针对性编程练习

Week 9 Lesson Plan: Review & Intervention

Lesson 1 (40 min): Examination

- ☐ Formal conditions, CIE-style paper
- ☐ No bilingual glossary (simulate exam conditions)
- ☐ Students write pseudocode in answer booklet

Lesson 2 (40 min): Whole-Class Review

- ☐ Display mark scheme
- ☐ Common errors identified:
 - Missing ENDIF in nested IFs
 - Wrong loop bounds (off-by-one)
 - Not returning value from function
 - Missing declarations

— Incorrect file closing

☐ Model answers shown

Lesson 3 (40 min): Differentiated Intervention

- ☐ Group A: Re-teach fundamentals with extra scaffolding; simplified tasks
- ☐ Group B: Targeted practice on weak construct (e.g., if recursion weak, extra recursion tasks)
- ☐ Group C: Advanced challenges — efficient algorithms, additional features, real-world scenarios
- ☐ 分层干预：根据评估结果分组进行针对性编程辅导

Weeks 1–9 Common Misconceptions Summary

1. Variables: Forgetting to declare before use. Using = instead of ← for assignment.
2. Selection: Missing ENDIF. Using = for comparison in pseudocode (CIE uses = for both, context determines).
3. Iteration: Infinite loops (forgetting to update counter). Off-by-one errors.
4. Subroutines: Confusing procedures and functions. Not understanding scope.
5. Recursion: Forgetting base case. Not understanding stack growth.
6. Arrays: Index out of bounds. Confusing 1-based vs 0-based indexing.
7. Records: Incorrect dot notation (Class[1].Name not Class.Name[1]).
8. Files: Not closing files. Reading past EOF.
9. Strings: Confusing 1-based indexing for substring. Forgetting LENGTH function.
10. General: Insufficient comments. Unmeaningful variable names. No input validation.

Address each explicitly with examples and practice.

Victoria Park Academy Shenzhen — A-Level Computer Science Curriculum 2024–2026